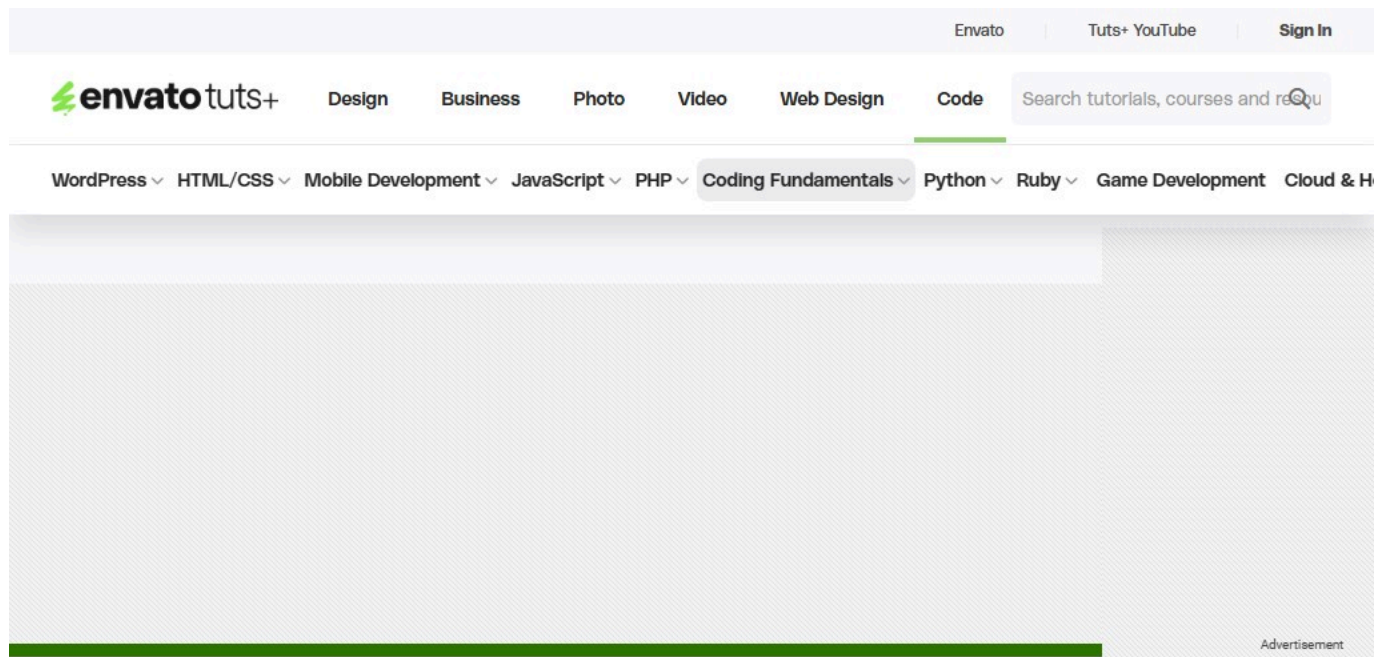


How to Begin With Test-Driven Development in Python

Tuesday, August 5, 2025 10:56 AM

Source: <https://code.tutsplus.com/beginning-test-driven-development-in-python--net-30137t>



Home > Code > Coding Fundamentals > Testing

How to Begin With Test-Driven Development in Python



David Sale

Feb 28, 2023 • 18 min read



Coding Fundamentals

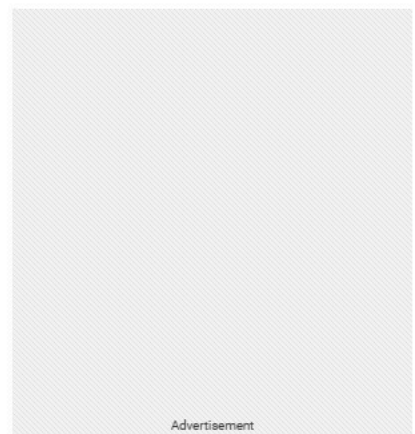
Testing

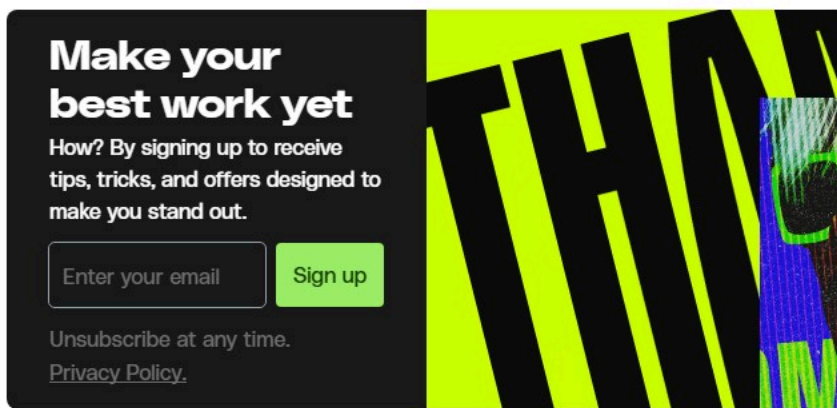
Python

Test-driven development (TDD) is a concept that's had a lot of attention in recent years. It is a practice of baking your tests right into your everyday coding, rather than leaving them as an afterthought.

In this post, I will introduce the core concepts of TDD.

The whole process is very simple to get to grips with, and it shouldn't take too long before you wonder how you were able to get anything done before! There are huge gains to be made from TDD—namely, the quality of your code improving, but also clarity and focus on what it is that you are trying to achieve, and the way in which you will achieve it. TDD also works seamlessly with agile development and can best be utilized when pair programming, as you will see later on.





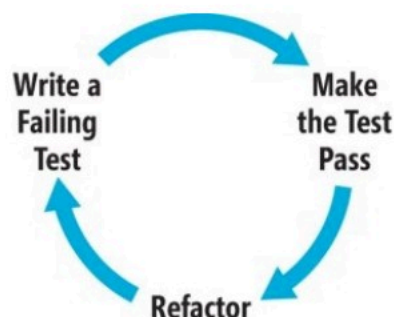
In this tutorial, I will introduce the core concepts of TDD and will provide examples in Python, using the nosetests unit-testing package. I will additionally offer some alternative packages that are also available within Python.

What Is Test-Driven Development?

TDD, in its most basic terms, is the process of implementing code by writing your tests first, seeing them fail, and then writing the code to make the tests pass. You can then build upon this developed code by appropriately altering your test to expect the outcome of additional functionality, before writing the code to make it pass again.

You can see that TDD is very much a cycle, with your code going through as many iterations of tests, writing, and development as necessary, until the feature is finished. Implementing these tests before you write the code brings out a natural tendency to *think* about your problem first. While you start to construct your test, you have to think about the way you design your code. What will this method return? What if we get an exception here? And so on.

Developing in this way means you consider the different routes through the code and cover these with tests as needed. This approach allows you to escape the trap that many developers fall into (myself included): diving into a problem and writing code exclusively for the first solution you need to handle.



The process can be defined as:

- write a failing unit test
- make the unit test pass
- refactor

Repeat this process for every feature as necessary.

Agile Development With Test-Driven Development

TDD is a perfect match for the ideals and principles of Agile Development, with a focus on delivering incremental updates to a product with true quality, as opposed to quantity. The confidence in your individual units of code that unit testing provides means that you meet this requirement to deliver quality, while eradicating issues in your production environments.

TDD comes into its own when pair programming, however. The ability to mix up your development workflow, when working as a pair as you see fit, is nice. For example, one person can write the unit test, see it pass, and then allow the other developer to write the code to make the test pass.

The roles can either be switched each time, each half day, or every day as you see fit. This means both parties in the pair are engaged, focused on what they are doing, and checking each other's work at every stage. This translates to a win in every sense with this approach, I think you'd agree.

TDD also forms an integral part of the Behaviour Driven Development process, which is again, writing tests up front, but in the form of acceptance tests. These ensure a feature behaves in the way you expect from end to end.

Syntax for Unit Testing

The main methods that we make use of in unit testing for Python are:

- `assert`: base assert allowing you to write your own assertions
- `assertEqual(a, b)`: check `a` and `b` are equal
- `assertNotEqual(a, b)`: check `a` and `b` are not equal
- `assertIn(a, b)`: check that `a` is in the item `b`
- `assertNotIn(a, b)`: check that `a` is not in the item `b`
- `assertFalse(a)`: check that the value of `a` is `False`
- `assertTrue(a)`: check that the value of `a` is `True`
- `assertIsInstance(a, TYPE)`: check that `a` is of type `TYPE`
- `assertRaises(ERROR, a, args)`: check that when `a` is called with `args`, it raises `ERROR`

There are certainly more methods available to us, which you can view—see the [Python Unit Test Docs](#)—but, in my experience, the ones listed above are among the most frequently used. We will make use of these within our examples below.

Installing and Using Python's Nose

Before starting the exercises below, you will need to install the `nosetest` test runner package. Installation of the `nosetest` runner is straightforward, following the standard `pip` install pattern. It's also usually a good idea to work

on your projects using `virtualenv`s, which keeps all the packages you use for various projects separate. If you are unfamiliar with `pip` or `virtualenv`'s, you can find documentation on them here: [VirtualEnv](#), [PIP](#).

The `pip` install is as easy as running this line:

```
1 | pip install nose
```

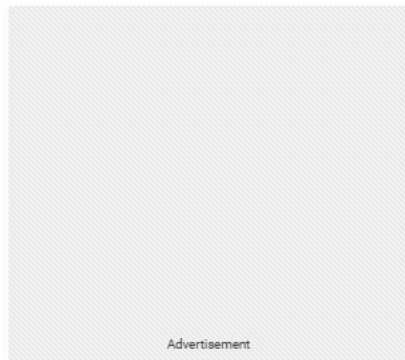
Once installed, you can execute a single test file.

```
1 | nosetests example_unit_test.py
```

Or execute a suite of tests in a folder.

```
1 | nosetests /path/to/tests
```

The only standard you need to follow is to begin each test's method with `test_` to ensure that the `nosetest` runner can find your tests!



Options

Some useful command-line options that you may wish to keep in mind include:

- `-v`: gives more verbose output, including the names of the tests being executed.
- `-s` OR `-nocapture`: allows output of print statements, which are normally captured and hidden while executing tests. Useful for debugging.
- `--nologcapture`: allows output of logging information.
- `--rednose`: an optional plugin, which can be downloaded [here](#), but provides colored output for the tests.
- `--tags=TAGS`: allows you to place a `@TAG` above a specific test to only execute those, rather than the entire test suite.

Example Problem and Test-Driven Approach

We are going to take a look at a really simple example to introduce both unit testing in Python and the concept of TDD. We will write a very simple calculator class, with `add`, `subtract`, and other simple methods as you would expect.

Following a TDD approach, let's say that we have a requirement for an `add` function, which will determine the sum of two numbers and return the output. Let's write a failing test for this.

In an empty project, create two Python packages, `app` and `test`. To make them Python packages (and thus support importing of the files in the tests later on), create an empty file called `__init__.py` in each directory. This is Python's standard structure for projects and must be followed to allow items to be imported across the directory structure. For a better understanding of this structure, you can refer to the [Python packages](#) documentation. Create a file named `test_calculator.py` in the test directory with the following contents.

```
1 import unittest
2
3 class TddInPythonExample(unittest.TestCase):
4     def test_calculator_add_method_returns_correct_result(self):
5         calc = Calculator()
6         result = calc.add(2,2)
7         self.assertEqual(4, result)
```

Writing the test is fairly simple.

- First, we import the standard `unittest` module from the Python standard library.
- Next, we need a class to contain the different test cases.
- Finally, a method is required for the test itself, with the only requirement being that it is named with "test_" at the beginning, so that it may be picked up and executed by the `nosetest` runner, which we will cover shortly.

With the structure in place, we can then write the test code. We initialize our calculator so that we can execute the methods on it. Following this, we can then call the `add` method which we wish to test, and store its value in the variable, `result`. Once this is complete, we can then make use of `unittest`'s `assertEqual` method to ensure that our calculator's `add` method behaves as expected.

Now you will use the `nosetest` runner to execute the test. You could execute the test using the standard `unittest` runner, if you wish, by adding the following block of code to the end of your test file.

```
1 if __name__ == '__main__':
2     unittest.main()
```

This will allow you to run the test using the standard way of executing Python files, `python test_calculator.py`. However, for this tutorial, you will use the `nosetest` runner, which has some nice features, such as executing nose tests against a directory and running all tests.

```
1 $ nosetests test_calculator.py
2 E
3 =====
4 ERROR: test_calculator_add_method_returns_correct_result (test.test_calcula
5 -----
6 Traceback (most recent call last):
7   File "/Users/user/PycharmProjects/tdd_in_python/test/test_calculator.py",
8     calc = Calculator()
9 NameError: global name 'Calculator' is not defined
10 -----
```

```
11 Ran 1 test in 0.001s
12 FAILED (errors=1)
```

From the output nosetest has given us, we can see that the problem relates to us not importing `Calculator`. That's because we haven't created it yet! So let's go and define our `Calculator` in a file named **calculator.py** under the **app** directory and import it:

app/calculator.py

```
1 class Calculator(object):
2     def add(self, x, y):
3         pass
```

test_calculator.py

```
1 import unittest
2 from app.calculator import Calculator
3
4 class TddInPythonExample(unittest.TestCase):
5     def test_calculator_add_method_returns_correct_result(self):
6         calc = Calculator()
7         result = calc.add(2,2)
8         self.assertEqual(4, result)
9
10 if __name__ == '__main__':
11     unittest.main()
```

Now that we have `Calculator` defined, let's see what nosetest indicates to us now:

```
1 $ nosetests test_calculator.py
2 F
3 =====
4 FAIL: test_calculator_add_method_returns_correct_result (test.test_calculator)
5 =====
6 Traceback (most recent call last):
7   File "/Users/user/PycharmProjects/tdd_in_python/test/test_calculator.py",
8     self.assertEqual(4, result)
9 AssertionError: 4 != None
10 =====
11 Ran 1 test in 0.001s
12 FAILED (failures=1)
```

So, obviously, our `add` method is returning the wrong value, as it doesn't do anything at the moment. Handily, nosetest gives us the offending line in the test, and we can then confirm what we need to change. Let's fix the method and see if our test passes now:

calculator.py

```
1 class Calculator(object):
2     def add(self, x, y):
3         return x+y
```

Output

```
1 $ nosetests test_calculator.py
2 .
3 -----
4 Ran 1 test in 0.000s
5 OK
```

Success! We have defined our `add` method, and it works as expected. However, there is more work to do around this method to ensure that we have tested it properly.

We have fallen into the trap of just testing the case we are interested in at the moment.

What would happen if someone were to add anything other than numbers? Python will actually allow for the addition of strings and other types, but in our case, for our calculator, it makes sense to only allow adding of numbers. Let's add another failing test for this case, making use of the `assertRaises` method to test if an exception is raised here:

`test_calculator.py`

```
1 import unittest
2 from app.calculator import Calculator
3
4 class TddInPythonExample(unittest.TestCase):
5     def setUp(self):
6         self.calc = Calculator()
7     def test_calculator_add_method_returns_correct_result(self):
8         result = self.calc.add(2, 2)
9         self.assertEqual(4, result)
10    def test_calculator_returns_error_message_if_both_args_not_numbers(self):
11        self.assertRaises(ValueError, self.calc.add, 'two', 'three')
12
13 if __name__ == '__main__':
14     unittest.main()
```

You can see above that we added the test and are now checking for a `ValueError` to be raised if we pass in strings. We could also add more checks for other types, but for now, we'll keep things simple. You may also notice that we've made use of the `setUp()` method. This allows us to put things in place before each test case. So, as we need our `Calculator` object to be available in both test cases, it makes sense to initialize this in the `setUp` method. Let's see what nosetest indicates to us now:

```
1 $ nosetests test_calculator.py
2 .F
3 =====
4 FAIL: test_calculator_returns_error_message_if_both_args_not_numbers (test_
5 -----
6 Traceback (most recent call last):
7   File "/Users/user/PycharmProjects/tdd_in_python/test/test_calculator.py",
8     self.assertRaises(ValueError, self.calc.add, 'two', 'three')
9 AssertionError: ValueError not raised
10 -----
11 Ran 2 tests in 0.001s
12 FAILED (failures=1)
```

Clearly, `nosetests` indicates to us that we are not raising the `ValueError` when we expect to be. Now that we have a new failing test, we can code the solution to make it pass.

```
1 class Calculator(object):
2     def add(self, x, y):
```

```

3     number_types = (int, float, complex)
4     if isinstance(x, number_types) and isinstance(y, number_types):
5         return x + y
6     else:
7         raise ValueError

```

From the code above, you can see that we've made a small addition to check the types of the values and whether they match what we want. One approach to this problem could mean that you follow [duck typing](#) and simply attempt to use it as a number, and try/except the errors that would be raised in other cases. The above is a bit of an edge case and means we must check before moving forward. As mentioned earlier, strings can be concatenated with the plus symbol, so we only want to allow numbers. Using the `isinstance` method allows us to ensure that the provided values can only be numbers.

To complete the testing, there are a couple of different cases that we can add. As there are two variables, it means that both could potentially not be numbers. Add the test case to cover all the scenarios.

```

1  import unittest
2  from app.calculator import Calculator
3
4  class TddInPythonExample(unittest.TestCase):
5      def setUp(self):
6          self.calc = Calculator()
7      def test_calculator_add_method_returns_correct_result(self):
8          result = self.calc.add(2, 2)
9          self.assertEqual(4, result)
10     def test_calculator_returns_error_message_if_both_args_not_numbers(self):
11         self.assertRaises(ValueError, self.calc.add, 'two', 'three')
12     def test_calculator_returns_error_message_if_x_arg_not_number(self):
13         self.assertRaises(ValueError, self.calc.add, 'two', 3)
14     def test_calculator_returns_error_message_if_y_arg_not_number(self):
15         self.assertRaises(ValueError, self.calc.add, 2, 'three')
16
17
18  if __name__ == '__main__':
19      unittest.main()

```

When we run all these tests now, we can confirm that the method meets our requirements!

```

1  $ nosetests test_calculator.py
2  ....
3  -----
4  Ran 4 tests in 0.001s
5  OK

```

Other Unit Test Packages

Advertisement

py.test

This is a similar test runner to `nosetest`, which makes use of the same conventions, meaning that you can execute your tests in either of the two. A nice feature of `pytest` is that it captures your output from the test at the bottom in a separate area, meaning you can quickly see anything printed to the command line (see below). I've found `pytest` to be useful when executing single tests, as opposed to a suite of tests.

To install the `pytest` runner, follow the same pip install procedure that you followed to install `nosetest`. Simply execute `$ pip install pytest`, and it will grab the latest version and install it on your machine. `pytest` allows you to run your tests by simply issuing the command `pytest` in the top-level project folder.

```
1  pytest
2  ===== test session starts =====
3  platform linux -- Python 3.9.12, pytest-7.2.1, pluggy-1.0.0
4  rootdir: /home/vaati/Desktop/TDD/test
5  collected 4 items
6
7  test_calculator.py ....
8
9  ===== 4 passed in 0.01s =====
```

An example of `pytest`'s output when printing from within your tests or code is shown below. This can be useful for quickly debugging your tests and seeing some of the data it is manipulating. NOTE: you will only be shown output from your code on errors or failures in your tests, otherwise `pytest` suppresses any output.

```
1  pytest
2  ===== test session starts =====
3  platform linux -- Python 3.9.12, pytest-7.2.1, pluggy-1.0.0
4  rootdir: /home/vaati/Desktop/TDD/test
5  collected 4 items
6
7  test_calculator.py F...
8
9  ===== FAILURES =====
10  _____ TddInPythonExample.test_calculator_add_method
11
12  self = <test.test_calculator.TddInPythonExample testMethod=test_calculator_
13
14  def test_calculator_add_method_returns_correct_result(self):
15      result = self.calc.add(2,2)
16  > self.assertEqual(4,result)
17  E   AssertionError: 4 != 0
18
19  test_calculator.py:9: AssertionError
20  ----- Captured stdout call -----
21  X is: 2
22  Y is: 2
23  Result is: 0
24  ===== short test summary info =====
25  FAILED test_calculator.py::TddInPythonExample::test_calculator_add_method_r
26  ===== 1 failed, 3 passed in 0.01s =====
```

UnitTest

Python's built-in `unittest` package that we have used to create our tests can actually be executed itself and gives nice output. This is useful if you don't wish

to install any external packages and keep everything pure to the standard library. To use this, simply add the following block to the end of your test file.

```
1 if __name__ == '__main__':
2     unittest.main()
```

Execute the test using `python calculator_tests.py`. Here is the output that you can expect:

```
1 $ python test/test_calculator.py
2 ....
3 -----
4 Ran 4 tests in 0.004s
5 OK
```

Debug Code With PDB

Often when following TDD, you will encounter issues with your code, and your tests will fail. There will be occasions where, when your tests do fail, it isn't immediately obvious why that is happening. In such instances, it will be necessary to apply some debugging techniques to your code to understand exactly how the code is manipulating the data and not getting the exact response or outcome that you expect.

Fortunately, when you find yourself in such a position, there are a couple of approaches you can take to understand what the code is doing and rectify the issue to get your tests passing. The simplest method, and one many beginners use when first writing Python code, is to add `print` statements at certain points in your code and see what they output when running tests.

Debug With Print Statements

If you deliberately alter our calculator code so that it fails, you can get an idea of how debugging your code will work. Change the code in the `add` method of `app/calculator.py` to subtract the two values.

```
1 class Calculator(object):
2     def add(self, x, y):
3         number_types = (int, float, complex)
4         if isinstance(x, number_types) and isinstance(y, number_types):
5             return x - y
6         else:
7             raise ValueError
```

When you run the tests now, the test which checks that your `add` method correctly returns four when adding two plus two fails, as it now returns 0. To check how it is reaching this conclusion, you could add some print statements to check that it is receiving the two values correctly and then check the output. This would then lead you to conclude the logic on the addition of the two numbers is incorrect. Add the following print statements to the code in `app/calculator.py`.

```
1 class Calculator(object):
```

```

1 class Calculator(object):
2     def add(self,x,y):
3         number_types = (int,float,complex)
4         if isinstance(x, number_types) and isinstance(y, number_types):
5             print('X is: {}'.format(x))
6             print('Y is: {}'.format(y))
7             result = x - y
8             print('Result is: {}'.format(result))
9             return result
10        else:
11            raise ValueError

```

Now, when you execute `nosetest` against the tests, it nicely shows you the captured output for the failing test, giving you a chance to understand the problem and fix the code to use addition rather than subtraction.

```

1 $ nosetests test/test_calculator.py
2 F...
3 =====
4 FAIL: test_calculator_add_method_returns_correct_result (test.test_calculator)
5 -----
6 Traceback (most recent call last):
7   File "/Users/user/PycharmProjects/tdd_in_python/test/test_calculator.py",
8     self.assertEqual(4, result)
9 AssertionError: 4 != 0
10 ----- >> begin captured stdout << -----
11 X is: 2
12 Y is: 2
13 Result is: 0
14 ----- >> end captured stdout << -----
15 -----
16 Ran 4 tests in 0.002s
17 FAILED (failures=1)

```

Advanced Debug With PDB

As you start to write more advanced code, print statements alone will not be enough or start to become tiresome to write all over the place and have to be cleaned up later. As the process of needing to debug has become commonplace when writing code, tools have evolved to make debugging Python code easier and more interactive.

One of the most commonly used tools is `pdb` (or Python Debugger). The tool is included in the standard library and simply requires adding one line where you would like to stop the program execution and enter into `pdb`, typically known as the "breakpoint". Using our failing code in the add method, try adding the following line before the two values are subtracted.

```

1 class Calculator(object):
2     def add(self, x, y):
3         number_types = (int, float, complex)
4         if isinstance(x, number_types) and isinstance(y, number_types):
5             import pdb; pdb.set_trace()
6             return x - y
7         else:
8             raise ValueError

```

If using `nosetest` to execute the test, be sure to execute using the `-s` flag, which tells `nosetest` not to capture standard output, otherwise your test will just hang and not give you the `pdb` prompt. Using the standard `unittest` runner and `pytest` does not require such a step.

With the `pdb` code snippet in place, when you execute the test now, the

execution of the code will break at the point at which you placed the `pdb` line, allowing you to interact with the code and variables that are currently loaded at the point of execution. When the execution first stops and you are given the `pdb` prompt, try typing `list` to see where you are in the code and what line you are currently at.

```
1  nosetests -s
2  > /Users/user/PycharmProjects/tdd_in_python/app/calculator.py(7)add()
3  -> return x - y
4  (Pdb) list
5      2         def add(self, x, y):
6          3             number_types = (int, float, complex)
7          4
8          5             if isinstance(x, number_types) and isinstance(y, number_type
9          6                 import pdb; pdb.set_trace()
10         7 ->         return x - y
11         8     else:
12         9         raise ValueError
13 [EOF]
14 (Pdb)
```

You can interact with your code, as if you were within a Python prompt, so try evaluating what is in the `x` and `y` variables at this point.

```
1  (Pdb) x
2  2
3  (Pdb) y
4  2
```

You can continue to "play" around with the code as you require to figure out what is wrong. You can type `help` at any point to get a list of commands, but here's the core set you will likely need:

- `n`: step forward to the next line of execution.
- `list`: show five lines either side of where you are currently executing to see the code involved with the current execution point.
- `args`: list the variables involved in the current execution point.
- `continue`: run the code through to completion.
- `jump <line number>`: run the code until the specified line number.
- `quit`/`exit`: stop `pdb`.

Conclusion

Test-Driven Development is a process that can be both fun to practice and hugely beneficial to the quality of your production code. Its flexibility in its application to anything from large projects with many team members right down to a small solo project means that it's a fantastic methodology to advocate to your team.

Whether pair programming or developing by yourself, the process of making a failing test pass is hugely satisfying. If you've ever argued that tests weren't necessary, hopefully this article has swayed your approach for future projects.

Make TDD a part of your daily workflow today!



By David Sale

Like

Every type of asset, for any type of project

Plus all the leading gen-AI tools

Start creating

Unlimited Downloads From \$16.50/month

Get access to over one million creative assets on Envato.

Over 9 Million Digital Assets

Everything you need for your next creative project.

Create Beautiful Logos, Designs & Mockups in Seconds

Design like a professional without Photoshop.

- ENVATO TUTS+
- About Envato Tuts+
 - Terms of Use
 - Privacy
 - Cookies
 - Cookie Settings
 - Do not sell or share my personal information

- HELP
- FAQ
 - Help Center

23,006

Tutorials

552

Courses

17,964

Translations

Certified

Corporation